Chapter 1

Ordinary Differential Equations

In Exercise 8.2, we found the equilibrium point where a duck would float on water. This kind of problem is called *static* because it does not move. This chapter introduces *dynamic* problems, which involve things that change over time.

Also, you'll learn about a mathematical tool for describing physical systems, *differential equations*, and two computational tools for solving them, Euler's method and `ode45`.

But first I have a quick suggestion about organizing code in files.

1.1 Functions and Files

So far we've only put one function in each file. It's also possible to put more than one function in a file, but only the first one, the *main function*, can be called from the Command Window. The other *local functions* can be called from anywhere inside the file, but not from the Command Window or any other file.

Keeping multiple functions in one file is convenient, but it makes debugging difficult because you can't call local functions from the Command Window.

To help with this problem, I often use the top-level function to develop and test my local functions. For example, to write a program for Exercise 8.2, I would create a file named *duck.m* and start with a main function named `duck` that takes no input variables and returns no output value.

Then I would write a function named `error_func` to evaluate the error function for `fzero`. To test `error_func`, I would call it from `duck` and then call `duck` from the Command Window.

Here's what my first draft might look like:

```
function res = duck()
    error = error_func(10)
end
```

```
function res = error_func(d)
    rho = 0.3;      % density in g / cm^3
    r = 10;         % radius in cm
    res = d;
end
```

This program is not complete, but it is enough code to test. Once this program is working, I would finish writing `error_func`. And once I'd finished and tested `error_func`, I would modify `duck` to use `fzero`.

This problem might only require two functions, but if there were more, I could write and test them one at a time and then combine them into a working program.

Now, let's get back to differential equations.

1.2 Differential Equations

A *differential equation (DE)* is an equation that describes the derivatives of an unknown function. "Solving a DE" means finding a function whose derivatives satisfy the equation.

For example, suppose we would like to predict the population of yeast growing in a nutrient solution. Assume that we know the initial population is 5 billion yeast cells. When yeast grow in particularly yeast-friendly conditions, the rate of growth at any point in time is proportional to the current population. If we define $y(t)$ to be the population at a time t, we can write the following equation for the rate of growth:

$$\frac{dy}{dt}(t) = ay(t)$$

where $\frac{dy}{dt}(t)$ is the derivative of $y(t)$ and a is a constant that characterizes how quickly the population grows. This equation is *differential* because it relates a function to one of its derivatives.

It is an *ordinary differential equation (ODE)* because all the derivatives involved are taken with respect to the same variable. If it related derivatives with respect to different variables (partial derivatives), it would be a *partial differential equation (PDE)*.

This equation is *first order* because it involves only first derivatives. If it involved second derivatives, it would be second order, and so on.

Lastly, it's *linear* because each term involves t, y, or dy/dt raised to the first power; if any of the terms involved products or powers of t, y, or dy/dt it would be *nonlinear*.

Now suppose we want to predict the yeast population in the future. We can do that using Euler's method.

1.3 Euler's Method

Here's a test to see if you're as smart as Leonhard Euler. Let's say you arrive at time (t) and measure the current population (y) and the rate of change (r). What do you think the population will be after some period of time Δt has elapsed?

If you said $y + r\Delta t$, congratulations! You just invented Euler's method.

This estimate is based on the assumption that r is constant, but in general it's not, so we only expect the estimate to be good if r changes slowly and Δt is small.

What if we want to make a prediction when Δt is large? One option is to break Δt into smaller pieces, called *time steps*. Then we can use the following equations to get from one time step to the next:

$$\begin{aligned} tt_{i+1} &= tt_i + dt \\ yy_{i+1} &= yy_i + \frac{df}{dt}(t)\, dt \end{aligned} \tag{1.1}$$

Here, tt_i is a vector of times when we estimate the value of y, and yy_i is the vector of estimates. For each index i, yy_i is an estimate of $y(tt_i)$.

If the rate doesn't change too fast and the time step isn't too big, Euler's method is accurate enough for most purposes.

1.4 Implementing Euler's Method

As an example we'll use Euler's method to solve Equation 1.1,

$$\frac{dy}{dt}(t) = ay(t)$$

with the initial condition $y(0) = 5$ cells and the growth parameter $a = 0.2$ cells/hr.

As a first step, create a file named *euler.m* with a main function and a local function:

```
function res = euler()
    tt(1) = 0;
    yy(1) = 5;
    r = rate_func(tt(1), yy(1))
end

function res = rate_func(t, y)
   a = 0.2;
   dydt = a * y;
   res = dydt;
end
```

In `euler` we initialize the initial conditions and then call `rate_func`, so called because it computes the rate of growth in the population.

After testing these functions, we can add code to `euler` to compute these difference equations:

$$\begin{aligned} tt_{i+1} &= tt_i + \Delta t \\ yy_{i+1} &= yy_i + r\Delta t \end{aligned}$$

where r is the rate of growth computed by `rate_func`. Listing 1.1 has the code we need:

Listing 1.1: A function implementing Euler's method

```
% Implement Euler's method. and show results
function res = euler()
    % Initial conditions
    tt(1) = 0;
    yy(1) = 5;
    % Time step
    dt = 0.1;

    % Loop for each time step
    for i=1:40
        r = rate_func(tt(i), yy(i));
        tt(i+1) = tt(i) + dt;
        yy(i+1) = yy(i) + r * dt;
    end
    plot(tt, yy)
    xlabel('Time [hours]')
    ylabel('Population [cells]')
end

function res = rate_func(t, y)
    %RATE_FUNC  Rate of growth at time (t) for population (y).
    a = 0.2;
    res = a * y;
end
```

Before the loop, we create two vectors, `tt` and `yy`, and set the first element of each with the initial conditions; `dt`, which is the size of the time steps, is 0.1 hours.

Inside the loop, we compute the growth rate based on the current time, `tt(i)`, and population, `tt(i)`. You might notice that the rate depends only on population, but we pass time as an input variable anyway, for reasons you'll see soon.

After computing the growth rate, we add an element both `tt` and `yy`, increasing their length by one. Then, when the loop exits, we plot `yy` as a function of `tt`, adding labels and units as good engineering practice.

If you run the code, you should get a plot of population over time, as shown in Figure 1.1.

As you can see, the population doubles in a little less than 4 hours. (You may recall for differential equations that for a linear, first-order system, the doubling time is $t_d = \ln(2)/r =$

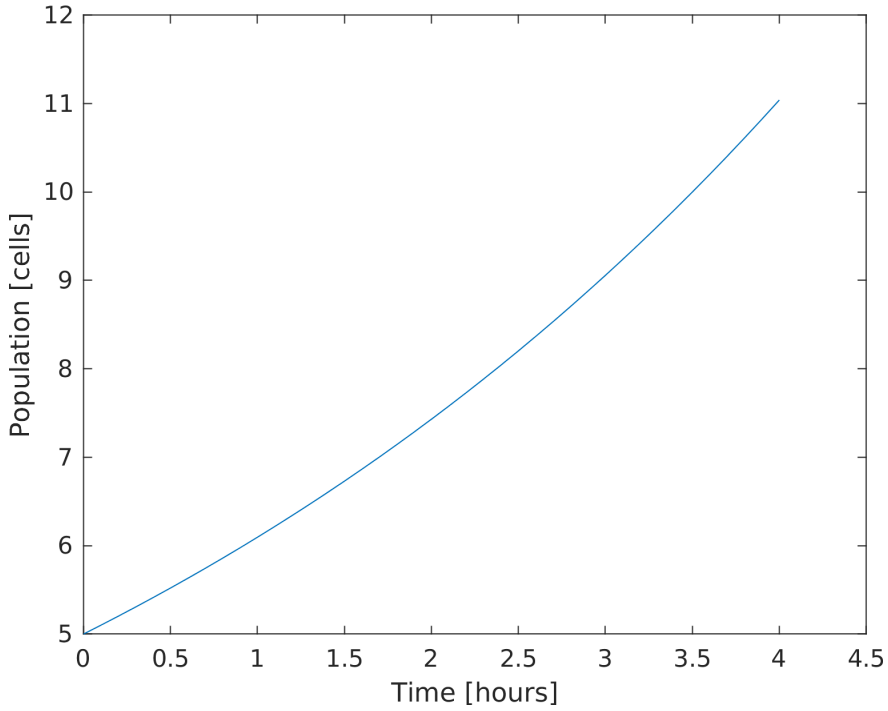


Figure 1.1: Solution to a simple differential equation by Euler's method

$\ln(2)/0.2 = 3.5\,\mathrm{s}$.)

1.5 Solving ODEs with ode45

A limitation of Euler's method is that it assumes that the derivative is constant between time steps, and that's not generally true. Fortunately, there are better methods that estimate the derivative between time steps, and they can be much more accurate.

MATLAB provides a function called `ode45` that implements one of these methods. In this section I'll explain how to use it; you can read more about how it works in Section 1.9 on page 13.

In order to use `ode45`, you have to write a function that evaluates dy/dt as a function of t and y. Fortunately, we already have one, called `rate_func`:

```
function res = rate_func(t, y)
   a = 0.2;
   dydt = a * y;
   res = dydt;
end
```

We can call `ode45` from the Command Window like this:

```
[T, Y] = ode45(@rate_func, [0, 4], 5);
plot(T, Y)
```

The first argument is a function handle, as we saw in Chapter 8. The second argument is the time interval where we want to evaluate the solution; in this case the interval is from $t = 0$ to $t = 4$ hours. The third argument is the initial population, 5 cells. The result is two vectors, a vector of time values for the solution and a vector of the same length with the solution.

The full example, as a live script, is as follows:

Compare Euler and ode45

First, define a local function to cacluate the rate of change.

```
function res = rate_func(t, y)
    %RATE_FUNC  Rate of growth at time (t) for population (y).
    a = 0.2;
    res = a * y;
end
```

Define Common Parameters

For both methods we'll define the initial conditions and time span

```
y_init = 5;
t_span = [0, 5];
```

Solve ODE via Euler's Method

Initialize the vectors for t and y

```
tt(1) = t_span(1);
yy(1) = y_init;
```

Time step

```
dt = 0.1;
```

Find the integer number of time steps in the time span.

```
n = ceil((t_span(2) - t_span(1))/dt);
```

Loop for each time step

```
for i=1:n
    r = rate_func(tt(i), yy(i));
    tt(i+1) = tt(i) + dt;
    yy(i+1) = yy(i) + r * dt;
end
```

Plot the results

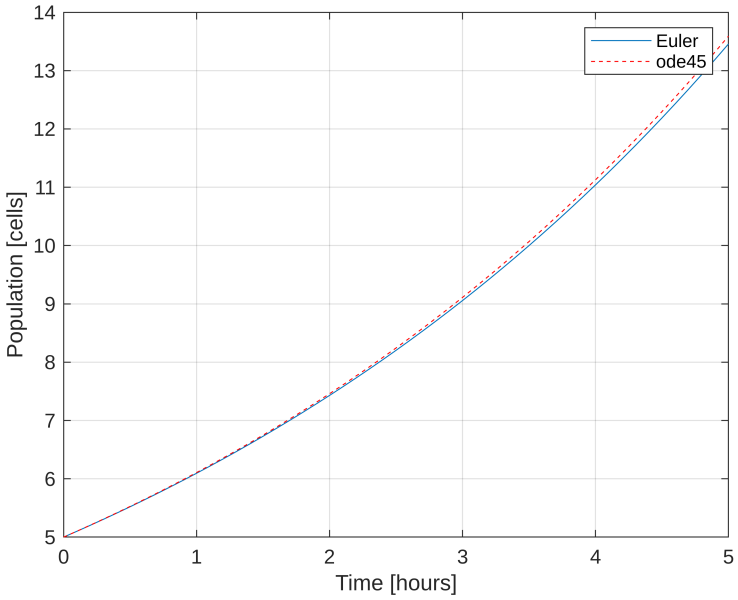
```
plot(tt, yy)
xlabel('Time [hours]')
ylabel('Population [cells]')
grid('on')
```

Solve ODE with ode45

```
[tout, yout] = ode45(@rate_func, t_span, y_init);
```

Overlay the new solution on the same plot. Add a legend to label each solution.

```
hold on;
plot(tout, yout, 'r--')
legend('Euler','ode45')
```



The solid line is the estimate we computed with Euler's method; the dashed line is the solution from `ode45`.

For the first 2–3 hours, the two solutions are visually indistinguishable. During the last hour, they diverge slightly; at 4 hours, the difference is less than 1 percent.

For many purposes, the difference between Euler's method and `ode45` is the least of our worries. In this example, we probably don't know the initial population with perfect accuracy or the growth constant, `a`. Also, the assumption that the growth rate only depends on population is probably not true. Any of these modeling errors could be bigger than 1 percent.

However, for some problems, Euler's method can be off by a lot more than 1 percent. In those cases `ode45` is almost always more accurate, for two reasons: first, it computes the rate function several times per time step; second, if the time step is too big, `ode45` can detect the problem and shrink the time step. For more details, see "How ode45 Works" on page 13.

1.6 Time Dependence

Looking at `rate_func` in the previous section, you might notice that it takes `t` as an input variable but doesn't use it. That's because the growth rate does not depend on time—bacteria don't know what time it is.

But rats do. Or, at least, they know what season it is. Suppose that the growth rate for rats depends on the current population *and* the availability of food, which varies over the course of the year. The differential equation might be something like

$$\frac{dy}{dt}(t) = ay(t)\,(1 - \cos(\omega t))$$

where t is time in days and $y(t)$ is the population at time t. Because the growth rate depends on time, this differential equation is *time dependent*.

The variables a and ω are *parameters*, which are values that quantify a physical aspect of the scenario. Parameters are often constants, but in some models they vary in time.

In this example, a characterizes the reproductive rate per day, and ω is the frequency of a periodic function that describes the effect of varying food supply on reproduction.

We'll use the values $a = 0.002\,\text{day}^{-1}$ and $\omega = 2\pi/365\,\text{rad/day}$ (one cycle per year). The growth rate is lowest at $t = 0$, on January 1, and highest at $t = 365/2$, on June 30.

Now we can write a function that evaluates the growth rate:

```
function res = rate_func(t, rats)
    %RATE_FUNC returns the growth rate at time (t) for population (rats)
    a = 0.002;
    omega = 2*pi / 365;
    res = a * rats * (1 - cos(omega * t));
end
```

To test this function, I put it in a file called *rats.m* with a main function called `rats`:

```
function res = rats_growth()
    t = 365/2;
    rats = 1000;
    res = rate_func(t, rats);
end
```

The main function assumes, for purposes of testing, that there are 1000 rats at $t = 365/2$ (June 30) and computes the growth rate under those conditions.

We can run the main function like this:

```
>> r = rat_growth()

r = 4
```

Under these conditions, the growth rate is 4 new rats per day.

Now that we've tested `rate_func`, we can use `ode45` to solve the differential equation. Here's how to call it from the main function in *rats.m*:

```
[tt, yy] = ode45(@rate_func, [0, 365], 1000)
plot(tt, yy)
```

The first argument is a function handle, again. The second argument is the interval we are interested in, a duration of one year, expressed in units of days. The third argument is the initial population, $y(0) = 1000$.

We can put this all together in the following program:

```
% Use ode45 to predict rat population growth.


% Initial condition
rats_init = 1000;
% Time span
t_span = [0, 365];

[tt, rats] = ode45(@rate_func, t_span, rats_init);
plot(tt, rats)
xlabel('Time (days)')
ylabel('Population (rats)')

% Return the final poplution number
res = rats(end);

function res = rate_func(t, rats)
    %RATE_FUNC returns the growth rate at time (t) for population (rats)
    a = 0.002;
```

```
    omega = 2*pi / 365;
    res = a * rats * (1 - cos(omega * t));
end
```

Figure 1.2 shows the results.

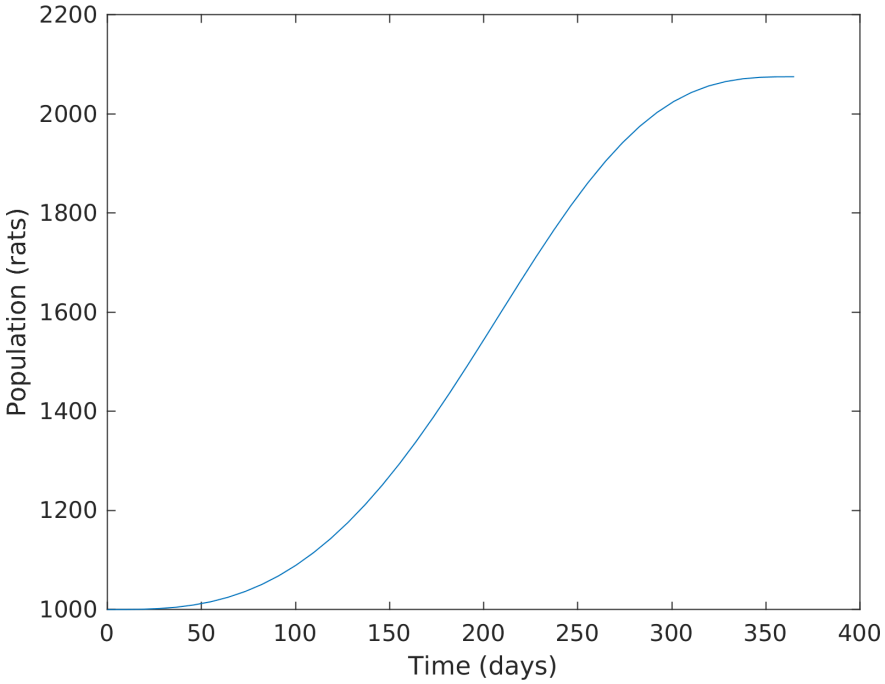


Figure 1.2: Solutions to a time-dependent differential equation by Euler's method and `ode45`

The population grows slowly during the winter, quickly during the summer, and then slowly again in the fall.

To see the population at the end of the year, you can display the last element of `rats`:

```
rats(end)
2.0751e+03
```

That's a little more than 2000 rats, so the population roughly doubles in one year.

The index here is `end`, which is a special word in MATLAB that means "the index of the last element." You can use it in an expression, so `Y(end - 1)` is the second-to-last element of `Y`.

1.7 What Could Go Wrong?

Don't forget the `@` on the function handle. If you leave it out, like:

```
[tt, yy] = ode45(rate_func, [0, 365], 1000)
```

MATLAB treats the first argument as a function call and calls `rate_func` without providing arguments. Then you get an error message:

```
Not enough input arguments.

Error in rats>rate_func (line 18)
    res = a * y * (1 - cos(omega * t));

Error in rats (line 6)
    [tt, yy] = ode45(rate_func, [0, 365], 1000);
```

Also, the rate function you write has to take two input variables, `t` and `y`, in that order, and return one output variable, `res`.

If you're working with a rate function like:

$$\frac{dy}{dt}(t) = ay(t)$$

you might be tempted to write this:

```
function res = rate_func(y)         % WRONG
    a = 0.002;
    res = a * y;
end
```

But that would be wrong. So very wrong. Why? Because when `ode45` calls `rate_func`, it provides two arguments. If you only take one input variable, you'll get an error. So you have to write a function that takes `t` as an input variable, even if you don't use it:

```
function res = rate_func(t, y)      % RIGHT
    a = 0.002;
    res = a * y;
end
```

Another common error is to write a function that doesn't make an assignment to the output variable. If you write something like:

```
function res = rate_func(t, y)
    a = 0.002;
    omega = 2*pi / 365;
    r = a * y * (1 - cos(omega * t));    % WRONG
end
```

and then call it from `ode45`, you get

```
Output argument "res" (and maybe others) not assigned during call
to "rate_func".
```

I hope these warnings save you some time debugging.

1.8 Labeling Axes

The plots in this chapter have labels on the axes, and one of them has a legend, but I didn't show you how to do that. Let's do it now.

The functions to label the axes are `xlabel` and `ylabel`:

```
xlabel('Time (hours)')
ylabel('Population (billions of cells)')
```

The function to generate a legend is `legend`:

```
legend('euler', 'ode45')
```

The arguments are the labels for the lines, in the order they were drawn. Usually the legend is in the upper-right corner, but you can move it by providing an optional argument called `Location`:

```
legend('euler', 'ode45', 'Location', 'northwest')
```

Finally, save the figures using `saveas`:

```
saveas(gcf,'runge.png','png')
```

The first argument is the figure we want to save; `gcf` is a MATLAB command that stands for "get current figure," which is the figure we just drew. The second argument is the filename. The extension specifies the format we want, which is a PNG (*.png*) image file. The third argument tells MATLAB what driver to use. The details aren't important, but `'png'` generates figures as color images.

1.9 How ode45 Works

According to the MATLAB documentation, `ode45` uses "an explicit Runge-Kutta formula, the Dormand-Prince pair." You can read about it at `https://greenteapress.com/matlab/runge`, but I'll give you a sense of it here.

The key idea behind all Runge-Kutta methods is to evaluate the rate function several times at each time step and use a weighted average of the computed slopes to estimate the value at the next time step. Different methods evaluate the rate function in different places and compute the average with different weights.

As an example, we'll solve the following differential equation:

$$\frac{dy}{dt}(t) = y \sin t$$

Given a differential equation, it's usually straightforward to write a rate function:

```
function res = rate_func(t, y)
    dydt = y * sin(t);
```

```
    res = dydt;
end
```

And we can use it like this:

```
    y0 = 1;
    tspan=[0 4];
    options = odeset('Refine', 1);
    [T, Y] = ode45(@rate_func, tspan, y0, options);
```

For this example we'll use `odeset` to set the `Refine` option to `1`, which tells `ode45` to return only the time steps it computes, rather than interpolating between them.

Now we can modify the rate function to plot the places where it gets evaluated:

```
function res = rate_func(t, y)
    dydt = y * sin(t);
    res = dydt;

    plot(t, y, 'ro')
    dt = 0.01;
    ts = [t t+dt];
    ys = [y y+dydt*dt];
    plot(ts, ys, 'r-')
end
```

When `rate_func` runs, it plots a red circle at each location and a short red line showing the computed slope.

Figure 1.3 shows the result; `ode45` computes 10 time steps (not counting the initial condition) and evaluates the rate function 61 times.

Figure 1.4 shows the same plot, zoomed in on a single time step. The dark squares at 0.8 to 1.2 show the values that were returned as part of the solution. The circles show the places where the rate function was evaluated.

We can see that `ode45` evaluates the rate function several times per time step, at several places between the end points. We can also see that most of the places where `ode45` evaluates the rate function are not part of the solution it returns, and they are not always good estimates of the solution. This is good to know when you are writing a rate function; you should not assume that the time and state you get as input variables will be part of the solution.

In a sense, the rate function is answering a hypothetical question: "*If* the state at a particular time has these particular values, what *would* the slope be?"

At each time step, `ode45` actually computes *two* estimates of the next value. By comparing them, it can estimate the magnitude of the error, which it uses to adjust the time step. If the error is too big, it uses a smaller time step; if the error is small enough, it uses a bigger

time step. Because `ode45` is *adaptive* in this way, it minimizes the number of times it calls the rate function to achieve a given level of accuracy.

1.10 Chapter Review

This chapter introduced *differential equations (DE)*, which are equations that describe the derivatives of an unknown function. In an *ordinary differential equation (ODE)*, all derivatives are taken with respect to the same variable, as opposed to a *partial differential equation (PDE)*, which includes derivatives with respect to more than one variable.

A *first-order DE* includes only first derivatives, and a *linear DE* includes no products or powers of the function and its derivatives. A differential equation is *time dependent* if the rate function depends on time.

When we solve a differential equation numerically, the *time step* is the interval in time between successive elements of the solution. A *parameter* is a value that appears in a model to quantify some physical aspect of the scenario being modeled.

Until now, we have only put one function in each M-file, but in this chapter we wrote a *main function*, which is the first function in an M-file, and a *local function*, which is any function in an M-file that is not the main function.

In the next chapter, we'll solve *systems* of ODEs, which are used to describe physical systems with multiple parts that interact. But first, here's an exercise where you can apply what you've learned so far.

1.11 Exercise

Before you go on, you might want to work on the following exercise.

Exercise 1.1. Suppose that you're given an 8-ounce cup of coffee at 90 °C. You have learned from bitter experience that the hottest coffee you can drink comfortably is 60 °C.

If the temperature of the coffee drops by 0.7 °C during the first minute, how long will you have to wait to drink your coffee?

You can answer this question with Newton's Law of Cooling (see `https://greenteapress.com/matlab/newton`):

$$\frac{dy}{dt}(t) = -k(y(t) - e)$$

where $y(t)$ is the temperature of the coffee at time t, e is the temperature of the environment, and k is a parameter that characterizes the rate of heat transfer from the coffee to the environment.

Let's assume that e is 20 °C and constant; that is, the coffee does not warm up the room.

Let's also assume k is constant. In that case, we can estimate it based on the information we have. If the temperature drops 0.7 °C during the first minute, when the coffee is 90 °C,

we can write

$$-0.7 = -k(90 - 20)$$

Solving this equation yields $k = 0.01$.

Here are some suggestions for getting started:

1. Create a file named *coffee.m* and write a function called `coffee` that takes no input variables. Put a simple statement like `x = 5` in the body of the function and invoke `coffee` from the Command Window.

2. Add a helper function called `rate_func` that takes `t` and `y` and computes dy/dt. In this case, `rate_func` does not actually depend on t; nevertheless, your function has to take t as the first input variable in order to work with `ode45`.

3. Test your function by adding a line like `rate_func(0, 90)` to `coffee`, then call `coffee` from the Command Window. Confirm that the initial rate is $-0.7\,^{\circ}\mathrm{C/min}$.

4. Once you get `rate_func` working, modify `coffee` to use `ode45` to compute the temperature of the coffee for 60 minutes. Confirm that the coffee cools quickly at first, then cools more slowly, and reaches room temperature after about an hour.

5. Plot the results and estimate the time when the temperature reaches $60\,^{\circ}\mathrm{C}$.

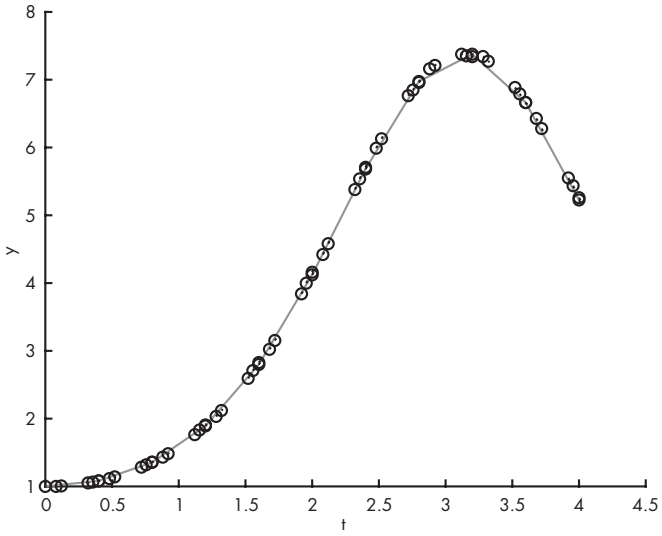


Figure 1.3: Points where `ode45` evaluates the rate function

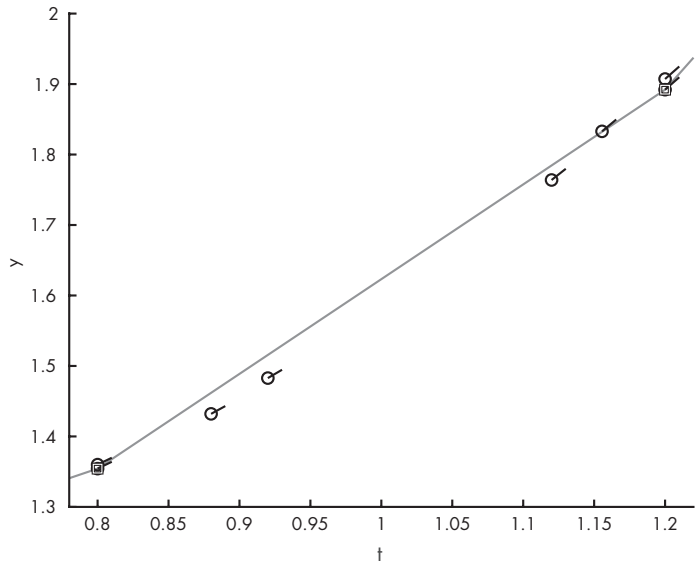


Figure 1.4: Points where `ode45` evaluates the rate function, zoomed in